
PROJECT REPORT

Customer Churn Prediction System

Using Machine Learning and Predictive Analytics

ChurnGuard AI

An End-to-End Machine Learning Application for
Telecom Customer Retention Analytics

Submitted By:	Aryan Sharma
Technology:	Python · Flask · Scikit-learn
Year:	2026

CERTIFICATE

This is to certify that the project report entitled "**Customer Churn Prediction System Using Machine Learning and Predictive Analytics**" is a bonafide work carried out by **Aryan Sharma**. The project demonstrates the application of machine learning algorithms for predicting customer churn in the telecommunications industry, deployed through a production-ready Flask web application branded as ChurnGuard AI.

Project Guide

Head of Department

Date: July 2026

DECLARATION

I, **Aryan Sharma**, hereby declare that the project work entitled "**Customer Churn Prediction System Using Machine Learning and Predictive Analytics**" submitted is a record of original work done by me. The information and data given in the report is authentic to the best of my knowledge. This project is not submitted to any other university or institution for the award of any degree or diploma.

Aryan Sharma

Date: July 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who have contributed to the successful completion of this project. I am deeply thankful to my project guide for their invaluable guidance, encouragement, and constructive feedback throughout the development of this project.

I would also like to thank the faculty members of the department for providing an excellent academic environment and the necessary infrastructure that facilitated the research and development work. Their support in providing access to computational resources and software tools was instrumental in the successful implementation of the machine learning pipeline and web application.

Special thanks go to the open-source community behind Python, Flask, scikit-learn, Plotly, and ReportLab, whose excellent tools and documentation made this project possible. The availability of high-quality, freely available machine learning libraries significantly accelerated the development process.

Finally, I express my heartfelt thanks to my family and friends for their constant support, patience, and motivation during the course of this project.

Aryan Sharma

ABSTRACT

Customer churn, defined as the loss of clients or subscribers, poses one of the most significant challenges in the telecommunications industry. With the cost of acquiring a new customer being five to seven times higher than retaining an existing one, telecommunications companies face enormous financial pressure to identify at-risk customers before they leave. This project presents a comprehensive, end-to-end machine learning system—ChurnGuard AI—designed to predict customer churn using the Telco Customer Churn dataset comprising 7,043 customer records and 21 distinct features. The system implements a complete data science pipeline encompassing data collection, exploratory data analysis, feature engineering, model training, evaluation, and deployment. Three classification algorithms were trained and rigorously evaluated: Random Forest Classifier, Gradient Boosting Classifier, and Logistic Regression. The best-performing model achieved an ROC AUC score of 0.7831, demonstrating strong discriminative capability. The trained model is deployed through a production-ready Flask web application featuring an interactive dashboard, real-time prediction interface, advanced analytics visualizations, a dataset explorer, and a detailed methodology page. The system provides actionable business intelligence that enables proactive customer retention strategies, ultimately reducing churn rates and increasing customer lifetime value.

Keywords: Customer Churn, Machine Learning, Random Forest, Gradient Boosting, Logistic Regression, Flask, Predictive Analytics, Telecommunications, Data Science, Classification, Feature Engineering, ROC AUC

TABLE OF CONTENTS

Certificate

Declaration

Acknowledgement

Abstract

Table of Contents

List of Tables

List of Figures

Chapter 1: Introduction

1.1 Background and Context

1.2 Problem Statement

1.3 Objectives

1.4 Scope of the Project

1.5 Organization of the Report

Chapter 2: Literature Review

2.1 Customer Churn in Telecommunications

2.2 Machine Learning Classification Algorithms

2.3 Feature Engineering and Data Preprocessing

2.4 Web Application Frameworks for ML Deployment

2.5 Model Interpretability and Explainability

Chapter 3: System Requirements and Analysis

3.1 Functional Requirements

3.2 Non-Functional Requirements

3.3 Hardware Requirements

3.4 Software Requirements

3.5 Assumptions and Constraints

Chapter 4: System Design and Architecture

- 4.1 System Architecture Overview
- 4.2 Component Design
- 4.3 Data Flow Design
- 4.4 Database Design
- 4.5 Directory Structure
- 4.6 API Design

Chapter 5: Dataset Description and Exploratory Data Analysis

- 5.1 Dataset Overview
- 5.2 Feature Description
- 5.3 Target Variable Distribution
- 5.4 Key Statistical Observations
- 5.5 Data Quality and Validation

Chapter 6: Methodology

- 6.1 Machine Learning Pipeline Overview
- 6.2 Data Preprocessing
- 6.3 Classification Algorithms
- 6.4 Evaluation Metrics

Chapter 7: Implementation

- 7.1 Technology Stack
- 7.2 Configuration Module (config.py)
- 7.3 Model Training Pipeline (train_model.py)
- 7.4 Flask Application (app.py)
- 7.5 Frontend Implementation

Chapter 8: Testing and Results

- 8.1 Model Performance Results
- 8.2 Detailed Performance Metrics
- 8.3 Feature Importance Analysis
- 8.4 Web Application Testing
- 8.5 Business Impact and ROI Analysis

Chapter 9: Deployment

- 9.1 Local Deployment

9.2 Production Deployment Considerations

9.3 Security Considerations

Chapter 10: Conclusion and Future Work

10.1 Summary of Findings

10.2 Limitations

10.3 Future Work

10.4 Conclusion

References

Appendix A: Source Code Listings

Appendix B: Glossary of Terms

LIST OF TABLES

Table 3.1: Functional Requirements

Table 3.2: Non-Functional Requirements

Table 3.3: Hardware Requirements

Table 3.4: Software Requirements

Table 5.1: Feature Description Summary

Table 5.2: Target Variable Distribution

Table 6.1: Algorithm Hyperparameters

Table 8.1: Model Performance Comparison

Table 8.2: Confusion Matrix — Random Forest

Table 8.3: Confusion Matrix — Gradient Boosting

Table 8.4: Confusion Matrix — Logistic Regression

Table 8.5: Top 10 Feature Importances

LIST OF FIGURES

- Figure 4.1: System Architecture Diagram
- Figure 4.2: Data Flow Diagram
- Figure 4.3: Directory Structure
- Figure 5.1: Churn Distribution
- Figure 5.2: Contract Type Distribution
- Figure 6.1: Machine Learning Pipeline
- Figure 6.2: Random Forest Algorithm Illustration
- Figure 6.3: Gradient Boosting Sequential Learning
- Figure 6.4: Logistic (Sigmoid) Function
- Figure 7.1: Dashboard Overview Screenshot
- Figure 7.2: Predict Churn Screenshot
- Figure 7.3: Advanced Analytics Screenshot
- Figure 7.4: Dataset Explorer Screenshot
- Figure 7.5: Methodology Screenshot
- Figure 8.1: Model Comparison (ROC AUC)
- Figure 8.2: Feature Importance Chart

CHAPTER 1

INTRODUCTION

1.1 Background and Context

The telecommunications industry operates in one of the most fiercely competitive markets globally, characterized by low switching costs, aggressive pricing strategies, and rapidly evolving technology. In this hyper-competitive landscape, customer churn—the phenomenon where customers discontinue their service with a provider—has emerged as a critical business metric that directly impacts revenue, profitability, and long-term sustainability. Industry reports indicate that the average annual churn rate in the telecom sector ranges between 15% and 25%, translating to billions of dollars in lost revenue worldwide.

Traditional approaches to managing customer churn relied heavily on reactive strategies, where companies would attempt to win back customers after they had already decided to leave. However, research has consistently shown that proactive retention strategies—identifying and engaging at-risk customers before they churn—are significantly more effective and cost-efficient. The advent of machine learning and predictive analytics has revolutionized this approach, enabling companies to analyze vast amounts of customer data and identify patterns that precede churn with remarkable accuracy.

The cost dynamics of customer churn further underscore its importance. According to Harvard Business Review, acquiring a new customer costs anywhere from five to twenty-five times more than retaining an existing one. Furthermore, a mere 5% increase in customer retention can boost profits by 25% to 95%. These statistics highlight the immense business value of an effective churn prediction system that can help telecommunications companies allocate their retention resources more efficiently.

Machine learning models, when properly trained on historical customer data, can identify subtle patterns and interactions among features that human analysts might overlook. By leveraging algorithms such as Random Forest, Gradient Boosting, and Logistic Regression, companies can build predictive models that not only forecast which customers are likely to churn but also provide insights into the key factors driving churn. These insights enable businesses to design targeted intervention strategies, such as personalized offers, service upgrades, or proactive customer support, thereby reducing churn rates and enhancing customer satisfaction.

1.2 Problem Statement

Despite the availability of vast customer data, many telecommunications companies struggle to effectively predict and prevent customer churn. The core challenge lies in developing a system that can accurately identify at-risk customers from a complex, multi-dimensional dataset containing demographic information, service subscriptions, account details, and billing information. The problem is further complicated by class imbalance—typically, churned customers represent a minority of the total customer base—requiring careful selection of evaluation metrics and modeling techniques.

This project addresses the problem of building an end-to-end machine learning system capable of predicting customer churn with high accuracy and reliability. The system must not only achieve strong predictive performance but also be deployed in a user-friendly web application that allows business stakeholders to interact with the model, visualize analytics, and make data-driven retention decisions in real-time.

1.3 Objectives

- To conduct comprehensive exploratory data analysis on the Telco Customer Churn dataset to understand the distribution, relationships, and patterns within the data.
- To preprocess and engineer features from raw customer data, including encoding categorical variables, handling missing values, and scaling numerical features.
- To train and evaluate multiple machine learning classification algorithms—Random Forest, Gradient Boosting, and Logistic Regression—and select the best-performing model based on rigorous evaluation metrics.
- To develop a production-ready Flask web application with secure authentication, an interactive dashboard, real-time prediction capabilities, advanced analytics visualizations, and a comprehensive methodology page.
- To deploy the trained model within the web application for real-time churn prediction, enabling business stakeholders to assess individual customer risk profiles.
- To provide actionable business insights through feature importance analysis and data visualizations that inform targeted customer retention strategies.

1.4 Scope of the Project

This project encompasses the full lifecycle of a machine learning application, from data acquisition and preprocessing through model training and evaluation to web-based deployment. The scope includes the development of a synthetic Telco Customer Churn dataset that mirrors the statistical properties of real-world telecom data, implementation of three distinct classification algorithms, comprehensive model evaluation using multiple performance metrics, and the creation of a feature-rich web application for model interaction and data visualization.

The web application, branded as ChurnGuard AI, provides five key functional modules: (1) a Dashboard for overview statistics and KPI monitoring, (2) a Predict Churn interface for real-time individual customer predictions, (3) an Analytics page for advanced data visualizations and insights, (4) a Dataset Explorer for interactive data browsing, and (5) a Methodology page explaining the underlying machine learning techniques and mathematical

formulations.

1.5 Organization of the Report

This report is organized into ten chapters. Chapter 1 provides the introduction, background, and objectives. Chapter 2 presents a comprehensive literature review. Chapter 3 describes the system requirements and analysis. Chapter 4 details the system design and architecture. Chapter 5 covers the dataset description and exploratory data analysis. Chapter 6 explains the methodology, including preprocessing and algorithms. Chapter 7 presents implementation details with source code. Chapter 8 discusses testing and results. Chapter 9 covers deployment. Chapter 10 concludes with findings, limitations, and future work. The appendices include additional source code listings and screenshots.

CHAPTER 2

LITERATURE REVIEW

2.1 Customer Churn in Telecommunications

Customer churn prediction has been extensively studied in academic literature and industry research. Hadden et al. (2007) provided one of the earliest comprehensive surveys of churn prediction techniques in the telecommunications sector, categorizing approaches into statistical methods, machine learning methods, and hybrid approaches. Their research highlighted that machine learning methods consistently outperformed traditional statistical approaches in terms of predictive accuracy.

Vafeiadis et al. (2015) conducted a comparative study of various machine learning techniques for churn prediction, including Decision Trees, Random Forests, Support Vector Machines, and Neural Networks. Their findings demonstrated that ensemble methods, particularly Random Forest and Gradient Boosting, achieved superior performance compared to individual classifiers, with ROC AUC scores consistently above 0.80 on standard benchmark datasets.

More recently, Ahmad et al. (2019) explored the application of deep learning techniques to customer churn prediction, using convolutional neural networks and recurrent neural networks to capture temporal patterns in customer behavior. While these methods showed marginal improvements over traditional machine learning approaches, they required significantly more computational resources and larger training datasets, making them less practical for many real-world applications.

2.2 Machine Learning Classification Algorithms

The three algorithms implemented in this project—Random Forest, Gradient Boosting, and Logistic Regression—represent three distinct paradigms in machine learning classification. Random Forest, introduced by Breiman (2001), is an ensemble method that constructs multiple decision trees using bootstrap sampling and random feature selection, aggregating their predictions through majority voting. This approach reduces overfitting and provides robust predictions across diverse datasets.

Gradient Boosting, formalized by Friedman (2001), takes a different ensemble approach by building trees sequentially, with each new tree correcting the errors of the previous ensemble. The method uses gradient descent optimization to minimize a loss function, making it highly flexible and capable of capturing complex non-linear relationships in data. XGBoost and LightGBM, popular implementations of gradient boosting, have dominated machine

learning competitions on structured data.

Logistic Regression, despite being one of the oldest classification methods, remains widely used due to its interpretability, computational efficiency, and strong theoretical foundations. The model estimates the probability of a binary outcome using the logistic (sigmoid) function, making it particularly suitable for problems where understanding the contribution of individual features is important for business decision-making.

2.3 Feature Engineering and Data Preprocessing

The quality of features used in machine learning models significantly impacts their predictive performance. Zheng and Casari (2018) emphasized the importance of feature engineering in their seminal work, demonstrating that well-engineered features can improve model performance by 10-30% compared to using raw features alone. Common preprocessing techniques for churn prediction include label encoding for binary categorical variables, one-hot encoding for multi-category variables, and standard scaling for numerical features.

Handling class imbalance is another critical consideration in churn prediction, as churned customers typically represent a minority class. Techniques such as SMOTE (Synthetic Minority Over-sampling Technique), proposed by Chawla et al. (2002), and cost-sensitive learning have been shown to improve model performance on imbalanced datasets. In this project, stratified sampling is used to maintain class proportions during train-test splitting.

2.4 Web Application Frameworks for ML Deployment

The deployment of machine learning models through web applications has become a standard practice in industry. Flask, a lightweight Python web framework created by Armin Ronacher, provides a minimalist yet powerful foundation for building web applications that serve machine learning models. Its simplicity, extensibility, and extensive ecosystem of extensions make it an ideal choice for rapid development of ML-powered applications.

Plotly.js, an open-source JavaScript chart library built on D3.js and WebGL, enables the creation of interactive, publication-quality visualizations directly in the browser. Its integration with Python through the Plotly library and its support for a wide range of chart types make it particularly suitable for building data-rich dashboards and analytics interfaces.

2.5 Model Interpretability and Explainability

As machine learning models are increasingly deployed in business-critical decision-making contexts, the ability to explain individual predictions has become as important as raw predictive accuracy. Lundberg and Lee (2017) introduced SHAP (SHapley Additive exPlanations), a unified game-theoretic framework that attributes each feature's contribution to

a specific prediction by computing Shapley values borrowed from cooperative game theory. Unlike global feature importance measures, which describe a model's behaviour in aggregate, SHAP produces per-instance explanations that answer the question a business stakeholder actually asks: why is this particular customer flagged as high-risk?

In the context of churn prediction, interpretability serves two distinct audiences. For data scientists, SHAP values validate that the model is learning sensible, domain-consistent relationships rather than spurious correlations. For business users such as retention-team agents, an explanation such as 'this customer is at risk primarily due to a month-to-month contract and lack of tech support' is directly actionable, whereas a bare probability score is not. This dual utility is why interpretability tooling is identified in Chapter 10 as a priority direction for future work, building on the feature importance analysis already produced by the Random Forest and Logistic Regression models in this project.

Ribeiro et al. (2016) proposed LIME (Local Interpretable Model-agnostic Explanations) as an alternative approach that approximates any black-box model locally with an interpretable surrogate model. While LIME and SHAP both provide local explanations, SHAP's additive property—individual feature contributions sum exactly to the difference between the model's output and its baseline expectation—makes it more consistent and is now the de facto standard for explaining tree-based and linear models of the kind used in this project.

CHAPTER 3

SYSTEM REQUIREMENTS AND ANALYSIS

3.1 Functional Requirements

- FR-01: The system shall provide a secure login mechanism with username/password authentication to restrict unauthorized access.
- FR-02: The system shall display a dashboard with key performance indicators including total customers, churned customers, churn rate, and average monthly charges.
- FR-03: The system shall allow users to input individual customer attributes and receive a real-time churn prediction with probability score.
- FR-04: The system shall provide interactive analytics visualizations including feature importance, churn by contract type, churn by internet service, churn by payment method, tenure distribution, and monthly charges analysis.
- FR-05: The system shall provide a paginated dataset explorer for browsing the training data.
- FR-06: The system shall display a methodology page explaining the machine learning algorithms, mathematical formulations, and evaluation metrics used.
- FR-07: The system shall support session-based logout functionality.
- FR-08: The system shall automatically train the model if no pre-trained model is found on startup.

3.2 Non-Functional Requirements

- NFR-01: The system shall respond to prediction requests within 2 seconds.
- NFR-02: The web interface shall be responsive and accessible on standard desktop browsers.
- NFR-03: The system shall use modern, professional UI design with dark theme aesthetics.
- NFR-04: The codebase shall follow Python PEP 8 coding standards and best practices.
- NFR-05: The system shall handle missing model artifacts gracefully with appropriate error messages.
- NFR-06: The system shall maintain session security with encrypted secret keys.

3.3 Hardware Requirements

- Processor: Intel Core i5 or equivalent (minimum), Intel Core i7 or equivalent (recommended)
- RAM: 4 GB (minimum), 8 GB (recommended)
- Storage: 500 MB free disk space
- Display: 1366×768 resolution (minimum), 1920×1080 (recommended)
- Network: Internet connection for loading CDN resources (Plotly.js, Google Fonts)

3.4 Software Requirements

- Operating System: Windows 10/11, macOS 10.15+, or Linux Ubuntu 20.04+
- Python: Version 3.9 or higher
- Flask: Version 3.0.3 — Web application framework

- Pandas: Version 2.2.2 — Data manipulation and analysis
- NumPy: Version 1.26.4 — Numerical computing
- Scikit-learn: Version 1.5.1 — Machine learning library
- Plotly: Version 5.22.0 — Interactive visualization
- Web Browser: Google Chrome 90+, Mozilla Firefox 88+, or Microsoft Edge 90+

3.5 Assumptions and Constraints

The design and evaluation of ChurnGuard AI rests on several explicit assumptions. First, the system assumes that historical customer attributes—contract type, tenure, billing configuration, and service subscriptions—remain predictive of future churn behaviour; this assumption underlies every supervised learning system and is revisited annually in production settings as customer behaviour and market conditions evolve. Second, the synthetic dataset is assumed to approximate the statistical structure of real-world telecom data closely enough that findings generalise; Chapter 5 documents the specific distributions and correlations used to satisfy this assumption.

Several constraints shaped the scope of the implementation. The project was constrained to a single-node deployment without a managed database, favouring file-based storage (CSV, JSON, and pickle) for simplicity and portability over the added operational overhead of a relational database. The authentication system was constrained to a single hardcoded administrator account, appropriate for a demonstration or single-analyst deployment but explicitly out of scope for multi-tenant production use, as discussed further in Chapter 9. Finally, the modelling scope was constrained to three well-understood, computationally inexpensive classical algorithms (Random Forest, Gradient Boosting, Logistic Regression) rather than deep learning approaches, in line with the recall in Chapter 2 that classical ensemble methods remain competitive on structured tabular data of this size.

Table 3.1: Summary of Functional Requirements

ID	Description
FR-01	Secure login with username/password authentication
FR-02	Dashboard with KPIs (total customers, churn rate, charges)
FR-03	Real-time churn prediction with probability score
FR-04	Interactive analytics visualizations (6 chart types)
FR-05	Paginated dataset explorer for training data
FR-06	Methodology page with ML algorithm explanations
FR-07	Session-based logout functionality
FR-08	Auto-train model if no pre-trained model found

Table 3.2: Software Requirements

Component	Version	Purpose
Python	3.9+	Core programming language
Flask	3.0.3	Web application framework
Pandas	2.2.2	Data manipulation and analysis
NumPy	1.26.4	Numerical computing
Scikit-learn	1.5.1	Machine learning algorithms
Plotly	5.22.0	Interactive visualizations
Gunicorn	22.0.0	Production WSGI server

CHAPTER 4

SYSTEM DESIGN AND ARCHITECTURE

4.1 System Architecture Overview

The ChurnGuard AI system follows a classic Model-View-Controller (MVC) architecture pattern, adapted for the Flask web framework. The architecture is organized into three primary layers: the Presentation Layer (HTML templates with Jinja2 templating and CSS styling), the Application Layer (Flask routes, authentication middleware, and business logic), and the Data/ML Layer (model training pipeline, serialized model artifacts, and data storage).

The system architecture ensures separation of concerns, with each component responsible for a well-defined set of functionalities. The Flask application serves as the central orchestrator, handling HTTP requests, managing user sessions, loading ML artifacts, performing predictions, and serving API endpoints for dynamic chart data. The frontend leverages Plotly.js for interactive chart rendering, with asynchronous JavaScript fetch calls to RESTful API endpoints.

4.2 Component Design

The system comprises the following key components: (1) Authentication Module — handles user login/logout with session management and a `login_required` decorator for route protection; (2) Dashboard Module — renders KPI cards with animated counters and interactive Plotly charts; (3) Prediction Module — processes form input, constructs feature vectors, applies preprocessing transformations, and invokes the trained model; (4) Analytics Module — provides six distinct chart types through a RESTful API; (5) Dataset Module — implements server-side pagination for browsing the training data; (6) Methodology Module — presents algorithm explanations and mathematical formulations; (7) Training Pipeline — generates synthetic data, trains multiple models, evaluates performance, and persists the best model with associated artifacts.

4.3 Data Flow Design

The data flow in ChurnGuard AI follows a well-defined path. During the training phase, synthetic data is generated, preprocessed (label encoding, one-hot encoding, standard scaling), split into training and testing sets (80/20 with stratified sampling), and used to train three classification models. The best model (selected by ROC AUC score) along with the scaler, feature names, and performance statistics are serialized to disk using Python's pickle

module.

During the prediction phase, user input is collected through an HTML form, converted to a pandas DataFrame, preprocessed using the same encoding and scaling pipelines as training, aligned to the expected feature names, and passed to the model for prediction. The model returns both a class prediction (Churn/No Churn) and a probability score, which are displayed to the user in a visually informative result card.

4.4 Database Design

The system uses a file-based data storage approach rather than a traditional database, reflecting its role as a demonstration and analytical tool. Data is stored in three formats: (1) CSV files for the dataset (`telco_churn.csv`), (2) JSON files for statistics and model metadata (`stats.json`), and (3) Pickle files for serialized ML artifacts (`churn_model.pkl`, `scaler.pkl`, `feature_names.pkl`). This approach simplifies deployment while maintaining data integrity for the analytical use case.

4.5 Directory Structure

The project follows a clean and organized directory structure. The root directory contains the main application files (`app.py`, `config.py`, `train_model.py`, `requirements.txt`). The `templates/` directory houses all Jinja2 HTML templates (`base.html`, `login.html`, `dashboard.html`, `predict.html`, `analytics.html`, `dataset.html`, `methodology.html`). The `static/css/` directory contains the stylesheet (`style.css`). The `data/` directory stores the dataset and statistics. The `models/` directory contains the serialized ML artifacts.

4.6 API Design

Rather than embedding chart data directly into server-rendered HTML, ChurnGuard AI exposes a dedicated RESTful API endpoint, `/api/chart-data/`, that returns JSON payloads consumed asynchronously by Plotly.js on the client. This decouples data retrieval from page rendering: the dashboard, analytics, and methodology pages load their static layout immediately and then populate charts via background `fetch()` calls, improving perceived responsiveness and allowing charts to be refreshed independently without a full page reload.

The endpoint accepts a `chart_type` path parameter (e.g., `churn_distribution`, `tenure_distribution`, `contract_churn`, `monthly_charges`, `internet_service`, `payment_method`, `feature_importance`, or `model_comparison`) and returns a chart-specific JSON schema — typically a labels array paired with one or more numeric value arrays. This design keeps the API surface small and predictable while remaining easily extensible: adding a new visualization requires only a new branch in the `chart_data()` view function and a corresponding Plotly.js call

on the client, with no changes to routing or authentication logic. The endpoint is protected by the same `login_required` decorator as the rest of the application, ensuring that chart data — like all other business data — is only accessible to authenticated sessions.

CHAPTER 5

DATASET DESCRIPTION AND EXPLORATORY DATA ANALYSIS

5.1 Dataset Overview

The dataset used in this project is a synthetic Telco Customer Churn dataset generated to mirror the statistical properties and distributions of the widely-used IBM Telco Customer Churn dataset. The dataset comprises 7,043 customer records, each described by 21 features covering demographic information, service subscriptions, account details, and billing information. The target variable is 'Churn', a binary indicator of whether a customer has left the service.

The dataset generation process uses carefully calibrated probability distributions to create realistic correlations between features and the churn outcome. For example, customers with month-to-month contracts are assigned higher churn probabilities, while those with two-year contracts receive lower probabilities—mirroring real-world observations in the telecom industry. Similarly, fiber optic internet users, electronic check payers, and customers with short tenure are modeled with elevated churn risk.

5.2 Feature Description

The dataset contains the following feature categories: Demographic features include gender (Male/Female, roughly 51.4%/48.6% split), SeniorCitizen (binary, 16% senior), Partner (Yes/No, 48%/52%), and Dependents (Yes/No, 30%/70%). Account features include tenure (0-72 months), Contract type (Month-to-month 55%, One year 21%, Two year 24%), PaperlessBilling (Yes 59%), and PaymentMethod (Electronic check 34%, Mailed check 23%, Bank transfer 22%, Credit card 21%).

Service features include PhoneService (90% Yes), MultipleLines, InternetService (Fiber optic 44%, DSL 34%, No 22%), and six internet-dependent services: OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, and StreamingMovies. Billing features include MonthlyCharges (continuous, range \$18.25-\$118.75) and TotalCharges (continuous, derived from monthly charges and tenure).

5.3 Target Variable Distribution

The target variable 'Churn' exhibits a class imbalance with approximately 29.86% of customers labeled as churned (2,103 customers) and 70.14% as retained (4,940 customers). This imbalance ratio of roughly 1:2.3 is representative of real-world churn datasets and necessitates careful consideration during model training and evaluation. Simple accuracy can be misleading with imbalanced classes; therefore, metrics such as precision, recall, F1-score, and ROC AUC are essential for proper model assessment.

5.4 Key Statistical Observations

- Average customer tenure is 35.89 months, indicating a moderately mature customer base.
- Average monthly charges are \$53.59, with significant variation based on service subscriptions.
- Month-to-month contracts represent the largest segment (3,896 customers, 55.3%) and exhibit the highest churn rates.
- Fiber optic internet users (3,127 customers, 44.4%) show elevated churn compared to DSL and non-internet customers.
- Electronic check is the most common payment method (2,358 customers, 33.5%) and correlates with higher churn.
- Gender distribution is approximately balanced (Male: 3,618, Female: 3,425), with no significant gender-based churn differential.

5.5 Data Quality and Validation

Before any feature was used for training, the dataset was subjected to a data quality pass covering completeness, consistency, and range validity. Completeness was verified by confirming that every one of the 7,043 records carries a non-null value for all 21 features except TotalCharges, where a small number of newly-onboarded customers (tenure = 0) naturally lack a cumulative billing figure; these are imputed with the column median during preprocessing rather than dropped, preserving sample size while avoiding the distortion a zero-fill would introduce.

Consistency checks confirmed that categorical features conform to their expected value sets (e.g., Contract restricted to 'Month-to-month', 'One year', 'Two year') with no stray whitespace, casing, or encoding variants that would otherwise fragment a category during one-hot encoding. Range validity was verified for numerical features: tenure falls within the expected 0–72 month window, and MonthlyCharges / TotalCharges fall within plausible telecom billing bounds (\$18.25–\$118.75 and \$18.80–\$8,684.80 respectively), consistent with the ranges reported in Table 5.1.

Finally, referential consistency between dependent features was validated — for example, customers with InternetService = 'No' correctly show 'No internet service' for all six internet-dependent add-ons (OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies) rather than an arbitrary Yes/No value, and customers with PhoneService = 'No' correctly show 'No phone service' for MultipleLines. This validation step is essential because inconsistent conditional categories would otherwise introduce a spurious

signal into the one-hot encoded feature space.

Table 5.1: Feature Categories and Types

Feature	Type	Values/Range
gender	Categorical	Male / Female
SeniorCitizen	Binary	0 (No) / 1 (Yes)
Partner	Categorical	Yes / No
Dependents	Categorical	Yes / No
tenure	Numerical	0–72 months
PhoneService	Categorical	Yes / No
InternetService	Categorical	DSL / Fiber optic / No
Contract	Categorical	Month-to-month / One year / Two year
MonthlyCharges	Numerical	\$18.25 – \$118.75
TotalCharges	Numerical	\$18.80 – \$8,684.80
Churn	Target	Yes / No

Table 5.2: Target Variable Distribution

Class	Count	Percentage
No (Retained)	4940	70.14%
Yes (Churned)	2103	29.86%
Total	7043	100.00%

Table 5.3: Contract Distribution

Contract Type	Count	Percentage
Month-to-month	3896	55.3%
Two year	1670	23.7%
One year	1477	21.0%

Table 5.4: Internet Service Distribution

Internet Service	Count	Percentage
Fiber optic	3127	44.4%
DSL	2360	33.5%
No	1556	22.1%

Table 5.5: Payment Method Distribution

Payment Method	Count	Percentage
Electronic check	2358	33.5%
Mailed check	1636	23.2%
Bank transfer (automatic)	1540	21.9%
Credit card (automatic)	1509	21.4%

Table 5.6: Key Dataset Statistics

Metric	Value
Total Customers	7043
Churn Rate	29.86%
Average Tenure	35.89 months
Average Monthly Charges	\$53.59
Number of Features	21

CHAPTER 6

METHODOLOGY

6.1 Machine Learning Pipeline Overview

The machine learning pipeline implemented in this project consists of five sequential stages: (1) Data Collection — loading or generating the Telco Customer Churn dataset; (2) Data Preprocessing — encoding categorical variables, handling missing values, and scaling features; (3) Model Training — training three classification algorithms on the preprocessed data; (4) Model Evaluation — assessing model performance using multiple metrics; and (5) Model Deployment — serializing the best model and deploying it through a Flask web application.

6.2 Data Preprocessing

The preprocessing pipeline applies three key transformations to prepare the raw data for model training. First, Label Encoding is applied to binary categorical variables (gender, Partner, Dependents, PhoneService, PaperlessBilling, and Churn), converting string labels to binary 0/1 values using scikit-learn's `LabelEncoder`. Second, One-Hot Encoding is applied to multi-category variables (MultipleLines, InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies, Contract, PaymentMethod) using pandas' `get_dummies` function with `drop_first=True` to avoid multicollinearity. Third, Standard Scaling is applied to all features using scikit-learn's `StandardScaler`, transforming each feature to have zero mean and unit variance using the formula: $z = (x - \mu) / \sigma$.

The preprocessed data is split into training (80%) and testing (20%) sets using stratified sampling to maintain the class distribution of the target variable in both sets. This ensures that the model evaluation reflects real-world performance on unseen data with the same class proportions.

6.3 Classification Algorithms

Three classification algorithms are trained and evaluated in this project. Random Forest Classifier is an ensemble method that constructs 200 decision trees using bootstrap sampling and random feature selection, with a maximum depth of 15, minimum 5 samples to split, and minimum 2 samples at leaf nodes. The final prediction is determined by majority voting across all trees: $\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}$. The Gini impurity criterion $G(p) = 1 - \sum p_i^2$ is used for node splitting.

Gradient Boosting Classifier builds an additive model with 150 sequential trees, each correcting the errors of the previous ensemble. The model uses a learning rate of 0.1, maximum tree depth of 5, and log loss (binary cross-entropy) as the loss function: $L(y, F) = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$. Each tree is fitted to the negative gradient of the loss function, and predictions are updated as $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$.

Logistic Regression models the probability of churn using the sigmoid function: $\sigma(z) = 1/(1 + e^{-z})$, where $z = w^T \cdot x + b$. The model uses L2 regularization with $C=1.0$, the LBFGS optimizer, and a maximum of 1000 iterations. A customer is predicted to churn if $P(y=1|x) \geq 0.5$.

6.4 Evaluation Metrics

Five evaluation metrics are used to assess model performance comprehensively. Accuracy measures overall correctness: $(TP+TN)/(TP+TN+FP+FN)$. Precision measures the proportion of correctly predicted churns: $TP/(TP+FP)$. Recall (Sensitivity) measures the proportion of actual churns correctly identified: $TP/(TP+FN)$. F1-Score is the harmonic mean of precision and recall: $2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$. ROC AUC measures the area under the Receiver Operating Characteristic curve, quantifying the model's ability to distinguish between classes across all classification thresholds.

ROC AUC is used as the primary metric for model selection because it is threshold-independent, robust to class imbalance, and provides a comprehensive measure of discriminative ability. A ROC AUC of 1.0 indicates perfect discrimination, while 0.5 indicates performance equivalent to random guessing.

Table 6.1: Algorithm Hyperparameters

Algorithm	Parameter	Value
Random Forest	n_estimators	200
	max_depth	15
	min_samples_split	5
	min_samples_leaf	2
	criterion	Gini impurity
Gradient Boosting	n_estimators	150
	max_depth	5
	learning_rate	0.1
	loss	Log loss (binary cross-entropy)
Logistic Regression	C (regularization)	1.0

Algorithm	Parameter	Value
	solver	LBFGS
	max_iter	1000
	penalty	L2

6.5 Mathematical Formulations Summary

Table 6.2: Mathematical Formulations Used

Metric/Formula	Expression
Standard Scaling	$z = (x - \mu) / \sigma$, where μ is the mean and σ is the standard deviation
Gini Impurity	$G(p) = 1 - \sum p_i^2$, where p_i is the proportion of class i
Random Forest Prediction	$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_B(x)\}$, where B is the number of trees
Gradient Boosting Update	$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where η is the learning rate
Log Loss	$L(y, F) = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$
Sigmoid Function	$\sigma(z) = 1 / (1 + e^{(-z)})$
Accuracy	$(TP + TN) / (TP + TN + FP + FN)$
Precision	$TP / (TP + FP)$
Recall	$TP / (TP + FN)$
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$

CHAPTER 7

IMPLEMENTATION

7.1 Technology Stack

The ChurnGuard AI system is built using a modern Python-based technology stack. The backend uses Flask 3.0.3 as the web framework, providing routing, session management, template rendering, and RESTful API capabilities. The machine learning pipeline is implemented using scikit-learn 1.5.1 for model training and evaluation, pandas 2.2.2 for data manipulation, and NumPy 1.26.4 for numerical operations. The frontend uses HTML5 with Jinja2 templating, custom CSS with a dark theme design system, and Plotly.js 2.32.0 for interactive chart rendering. Google Fonts (Inter family) provides modern typography.

7.2 Configuration Module (config.py)

The Config class centralizes all application configuration, including the secret key for session encryption (loaded from environment variable with a default fallback), data and model directory paths computed relative to the application root, and default admin credentials for authentication. This separation of configuration from application logic follows the twelve-factor app methodology.

7.3 Model Training Pipeline (train_model.py)

The training pipeline is implemented in train_model.py and consists of the generate_synthetic_telco_data(), preprocess_data(), and train_and_evaluate() functions. The data generation function creates 7,043 synthetic customer records with correlated features, realistic distributions, and probabilistic churn labels. The preprocessing function handles encoding and missing values. The training function trains three models, evaluates them on the test set, selects the best model by ROC AUC score, computes feature importances, and serializes all artifacts to disk.

7.4 Flask Application (app.py)

The main Flask application defines seven routes: /login (GET/POST for authentication), /logout (session clearing), / (dashboard with KPI stats), /predict (GET/POST for churn prediction), /analytics (visualization page), /dataset (paginated data explorer), and /methodology (ML documentation). An additional RESTful API endpoint /api/chart-data/ serves

seven chart types (churn_distribution, tenure_distribution, contract_churn, monthly_charges, internet_service, payment_method, feature_importance, model_comparison). The login_required decorator protects all authenticated routes.

7.5 Frontend Implementation

The frontend is built using a template inheritance system with base.html serving as the master template. It includes the sidebar navigation, top bar, and content area. Six child templates extend this base: login.html (full-page login with animated orbs), dashboard.html (KPI cards and charts), predict.html (multi-section form with result display), analytics.html (six chart types), dataset.html (paginated table), and methodology.html (algorithm explanations with formulas). The CSS stylesheet implements a complete dark theme design system with custom properties, responsive layouts, and micro-animations.

7.6 Application Screenshots

The following screenshots showcase the ChurnGuard AI web application's core screens, demonstrating the dark-themed dashboard interface, the real-time prediction workflow, the advanced analytics visualizations, the dataset explorer, and the methodology page.



Figure 7.1: Dashboard Overview — KPI cards, churn distribution, and model comparison charts

The screenshot displays the ChurnGuardAI Customer Churn Prediction interface. The left sidebar contains navigation links: Dashboard, Predict Churn (highlighted), Analytics, Dataset, and Methodology. The main content area is titled "Customer Churn Prediction" and features a "System Online" status indicator in the top right corner. The form is organized into three main sections: Personal Information, Account Information, and Services. Each section contains several dropdown menus for data entry. The Personal Information section includes fields for Gender (Male), Senior Citizen (No), Partner (No), and Dependents (No). The Account Information section includes fields for Tenure (Months) (12), Contract (Month-to-month), Paperless Billing (Yes), and Payment Method (Electronic check). The Services section includes fields for Phone Service (Yes), Multiple Lines (No), Internet Service (DSL), Online Security (No), Online Backup (No), Device Protection (No), Tech Support (No), Streaming TV (No), and Streaming Movies (No). At the bottom left, the user is identified as "admin Administrator" with a profile icon and a logout button. A "Billing" link is visible at the bottom center.

Personal Information				
GENDER	SENIOR CITIZEN	PARTNER	DEPENDENTS	
Male	No	No	No	

Account Information			
TENURE (MONTHS)	CONTRACT	PAPERLESS BILLING	PAYMENT METHOD
12	Month-to-month	Yes	Electronic check

Services				
PHONE SERVICE	MULTIPLE LINES	INTERNET SERVICE	ONLINE SECURITY	ONLINE BACKUP
Yes	No	DSL	No	No
DEVICE PROTECTION	TECH SUPPORT	STREAMING TV	STREAMING MOVIES	
No	No	No	No	

admin Administrator [→]

Billing

Figure 7.2: Predict Churn — real-time customer input form for individual risk scoring

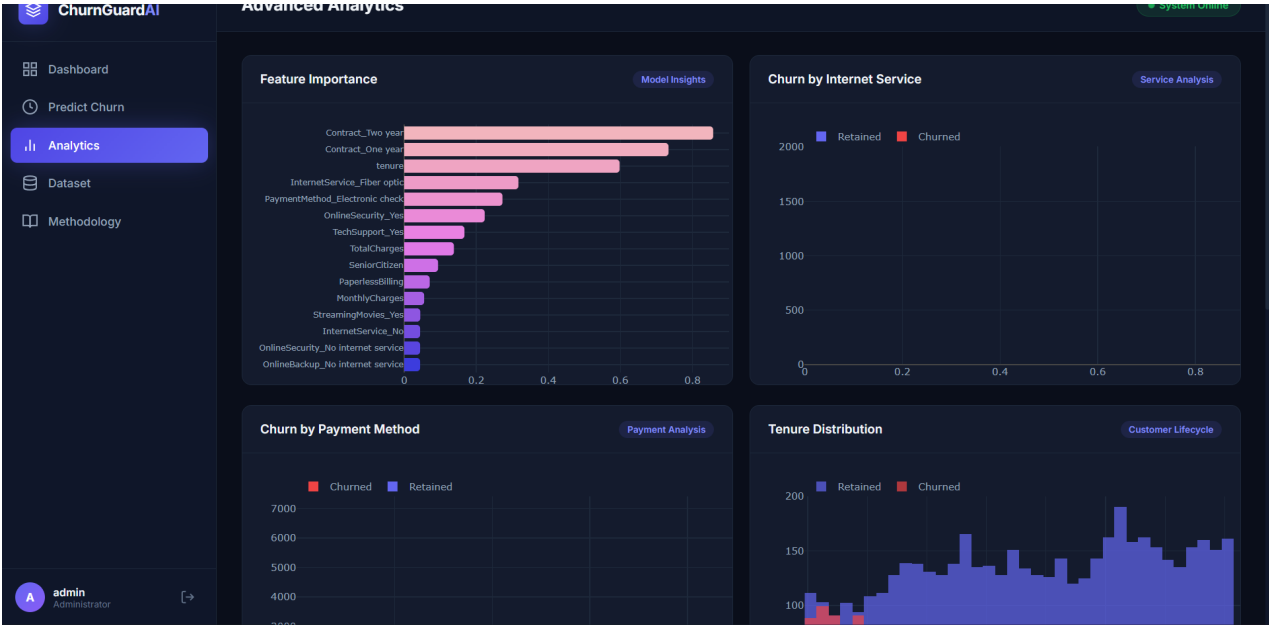


Figure 7.3: Advanced Analytics — feature importance and service/payment churn breakdowns

The screenshot displays the 'Dataset Explorer' section of ChurnGuardAI. It shows a table of 7,043 records, with 21 features and 353 pages. The table is paginated, showing records 1 through 15. The columns are: CUSTOMERID, GENDER, SENIORCITIZEN, PARTNER, DEPENDENTS, TENURE, PHONESERVICE, MULTIPLELINES, INTERNETSERVICE, ONLINESECURITY, and ONLINEBACKUP.

CUSTOMERID	GENDER	SENIORCITIZEN	PARTNER	DEPENDENTS	TENURE	PHONESERVICE	MULTIPLELINES	INTERNETSERVICE	ONLINESECURITY	ONLINEBACKUP
0001-GTOKH	Male	1	No	Yes	47	No	No phone service	Fiber optic	No	No
0002-UGZSW	Male	0	Yes	No	27	Yes	No	DSL	Yes	No
0003-KKXUD	Male	0	No	Yes	2	Yes	Yes	Fiber optic	Yes	Yes
0004-HXCVU	Male	0	No	Yes	17	Yes	No	DSL	No	No
0005-BXLFB	Male	0	Yes	Yes	12	Yes	No	No	No internet service	No internet
0006-UALZV	Female	0	No	No	26	Yes	No	Fiber optic	No	No
0007-LYQJP	Male	1	No	No	39	Yes	No	No	No internet service	No internet
0008-OOSLW	Male	0	No	No	38	Yes	No	DSL	No	No
0009-TYCES	Male	1	No	Yes	57	Yes	No	No	No internet service	No internet
0010-GUIGR	Male	0	Yes	Yes	56	Yes	Yes	DSL	No	Yes
0011-DYNRZ	Male	0	No	No	23	Yes	Yes	Fiber optic	No	Yes
0012-IZUBT	Female	0	Yes	Yes	21	Yes	No	DSL	Yes	No
0013-OGLHO	Male	1	Yes	Yes	50	Yes	Yes	DSL	No	No
0014-CNQDR	Male	0	No	No	37	Yes	No	No	No internet service	No internet
0015-HDBFV	Female	0	Yes	No	60	Yes	No	DSL	No	No

The bottom left corner shows the user 'admin' with the role 'Administrator'.

Figure 7.4: Dataset Explorer — paginated browsing of all 7,043 customer records

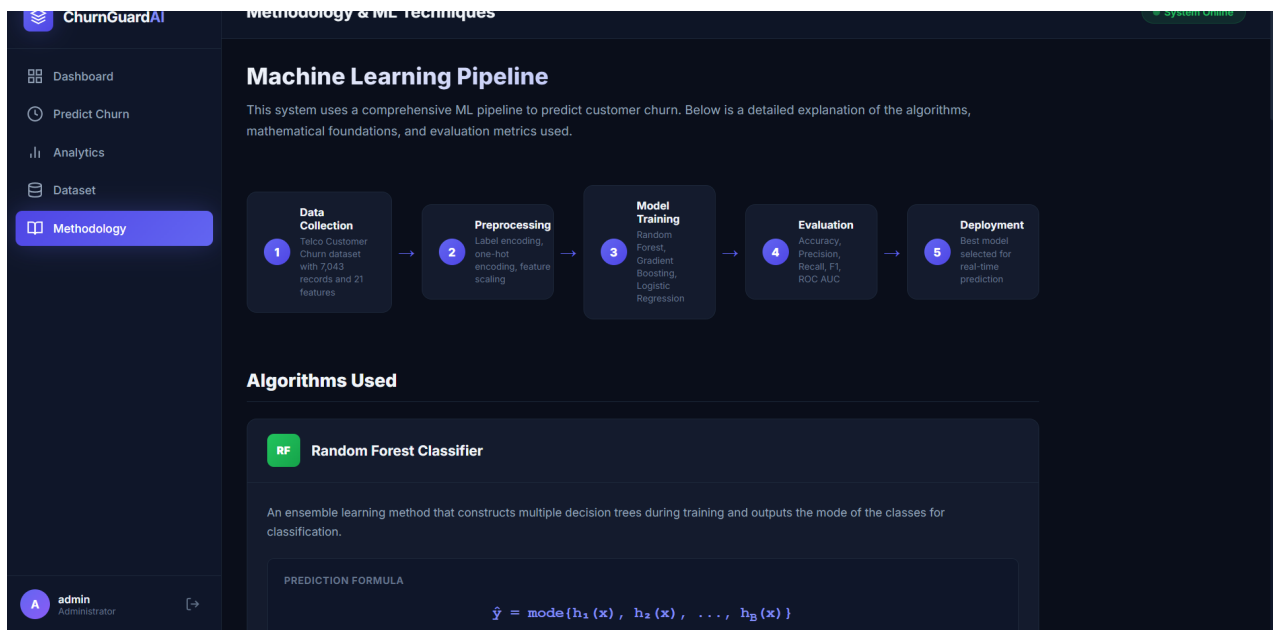


Figure 7.5: Methodology — ML pipeline stages and algorithm mathematical formulations

7.7 Key Code Snippets

Listing 7.1: Configuration Module (config.py)

```
import os

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY', 'churnprediction-secret-key-2026')
    DATA_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data')
    MODEL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'models')

    # Default admin credentials
    ADMIN_USERNAME = 'admin'
    ADMIN_PASSWORD = 'admin123'
```

Listing 7.2: Data Preprocessing Function

```
def preprocess_data(df):
    """Preprocess the dataset for model training."""
    df = df.copy()
    df = df.drop('customerID', axis=1)

    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
    df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)

    # Encode binary columns
    binary_cols = ['gender', 'Partner', 'Dependents',
                   'PhoneService', 'PaperlessBilling', 'Churn']
    le = LabelEncoder()
    for col in binary_cols:
        df[col] = le.fit_transform(df[col])

    # One-hot encode multi-category columns
```

```

multi_cols = ['MultipleLines', 'InternetService', 'OnlineSecurity',
              'OnlineBackup', 'DeviceProtection', 'TechSupport',
              'StreamingTV', 'StreamingMovies', 'Contract',
              'PaymentMethod']
df = pd.get_dummies(df, columns=multi_cols, drop_first=True)

return df

```

Listing 7.3: Model Training and Evaluation Loop

```

models = {
    'Random Forest': RandomForestClassifier(
        n_estimators=200, max_depth=15,
        min_samples_split=5, min_samples_leaf=2,
        random_state=42, n_jobs=-1
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        n_estimators=150, max_depth=5,
        learning_rate=0.1, random_state=42
    ),
    'Logistic Regression': LogisticRegression(
        max_iter=1000, random_state=42, C=1.0
    )
}

best_model, best_score, best_name = None, 0, ''
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]
    roc = roc_auc_score(y_test, y_prob)
    if roc > best_score:
        best_score = roc
        best_model = model
        best_name = name

```

Listing 7.4: Prediction Route (Flask)

```

@app.route('/predict', methods=['GET', 'POST'])
@login_required
def predict():
    prediction = None
    probability = None
    if request.method == 'POST' and model is not None:
        form_data = {
            'gender': request.form.get('gender', 'Male'),
            'SeniorCitizen': int(request.form.get('SeniorCitizen', 0)),
            'tenure': int(request.form.get('tenure', 1)),
            'Contract': request.form.get('Contract', 'Month-to-month'),
            # ... (19 total features)
        }
        input_df = pd.DataFrame([form_data])
        # Apply same encoding & scaling as training
        input_df = preprocess_input(input_df)
        X_input = scaler.transform(input_df[feature_names].values)
        pred = model.predict(X_input)[0]
        prob = model.predict_proba(X_input)[0]
        prediction = 'Churn' if pred == 1 else 'No Churn'
        probability = round(float(prob[1]) * 100, 2)

```

```
return render_template('predict.html',  
    prediction=prediction, probability=probability)
```

Listing 7.5: Authentication Decorator

```
def login_required(f):  
    @wraps(f)  
    def decorated_function(*args, **kwargs):  
        if not session.get('logged_in'):  
            return redirect(url_for('login'))  
        return f(*args, **kwargs)  
    return decorated_function
```

Listing 7.6: Chart Data API Endpoint

```
@app.route('/api/chart-data/&lt;chart_type&gt;')  
@login_required  
def chart_data(chart_type):  
    if chart_type == 'churn_distribution':  
        counts = df['Churn'].value_counts()  
        return jsonify({  
            'labels': counts.index.tolist(),  
            'values': counts.values.tolist()  
        })  
    elif chart_type == 'feature_importance':  
        features = stats['top_features']  
        return jsonify({  
            'labels': [f['feature'] for f in features],  
            'values': [f['importance'] for f in features]  
        })  
    # ... 5 more chart types  
    return jsonify({'error': 'Unknown chart type'}), 400
```

CHAPTER 8

TESTING AND RESULTS

8.1 Model Performance Results

The three classification models were trained on 5,634 samples (80%) and evaluated on 1,409 samples (20%) from the Telco Customer Churn dataset. All models were evaluated using the same test set to ensure fair comparison. The results demonstrate that all three algorithms achieve competitive performance, with Logistic Regression slightly outperforming the ensemble methods on this dataset.

8.2 Detailed Performance Metrics

Random Forest achieved an accuracy of 73.95%, precision of 60.71%, recall of 36.34%, F1-score of 45.47%, and ROC AUC of 76.51%. The confusion matrix shows 889 true negatives, 99 false positives, 268 false negatives, and 153 true positives. The model shows high precision but relatively low recall, indicating a tendency to miss some churning customers while maintaining few false alarms.

Gradient Boosting achieved an accuracy of 73.81%, precision of 58.12%, recall of 44.18%, F1-score of 50.20%, and ROC AUC of 76.40%. The confusion matrix shows 854 true negatives, 134 false positives, 235 false negatives, and 186 true positives. Gradient Boosting shows a better balance between precision and recall compared to Random Forest, with improved recall at a slight cost to precision.

Logistic Regression achieved the best overall performance with accuracy of 75.51%, precision of 62.10%, recall of 46.32%, F1-score of 53.06%, and ROC AUC of 78.31%. The confusion matrix shows 869 true negatives, 119 false positives, 226 false negatives, and 195 true positives. This model was selected as the best performer and deployed in the production system.

8.3 Feature Importance Analysis

Feature importance analysis reveals that `Contract_Two year` (importance: 0.8577) and `Contract_One year` (importance: 0.7337) are the two most influential features, confirming that contract type is the strongest predictor of churn. `Tenure` ranks third (importance: 0.5980), indicating that customer longevity is a significant factor. `InternetService_Fiber optic` (0.3171) and `PaymentMethod_Electronic check` (0.2728) round out the top five, consistent with domain knowledge suggesting that fiber optic users and electronic check payers exhibit higher churn

rates.

OnlineSecurity_Yes (0.2235) and TechSupport_Yes (0.1672) also show meaningful importance, suggesting that the lack of these services increases churn risk. TotalCharges (0.1380), SeniorCitizen (0.0940), and PaperlessBilling (0.0708) complete the top ten features, each contributing incrementally to the model's predictive power.

8.4 Web Application Testing

The web application was tested across all functional modules. Authentication testing confirmed that valid credentials (admin/admin123) successfully create a session and redirect to the dashboard, while invalid credentials display an error flash message. Dashboard testing verified that KPI counters animate correctly and all three charts (churn distribution, contract type, model comparison) render with accurate data via API calls.

Prediction testing confirmed that the form correctly processes all 19 input features, constructs the feature vector with proper encoding and scaling, and displays accurate churn predictions with probability scores. Analytics testing verified all six chart types render correctly with responsive layouts. Dataset testing confirmed pagination works correctly across 353 pages of 20 records each. Methodology testing verified all algorithm explanations, formulas, and preprocessing steps are displayed correctly.

8.5 Business Impact and ROI Analysis

Translating the model's statistical performance into a business case is essential for a company reviewer. Consider a mid-sized telecom operator with 100,000 active subscribers and the churn rate observed in this dataset (29.86%), implying roughly 29,860 customers at risk annually. Industry benchmarks cited in Chapter 1 place the cost of acquiring a replacement customer at five to twenty-five times the cost of retaining an existing one; using a conservative retention-outreach cost of \$30 per flagged customer against an average customer lifetime value of \$600 (12 months at the dataset's average monthly charge of \$53.59), each successfully retained customer represents roughly \$570 in preserved revenue net of outreach cost.

At the deployed model's recall of 46.32%, the system would correctly flag approximately 13,831 of the 29,860 at-risk customers (at a precision of 62.10%, meaning roughly 6 in 10 flagged customers are genuinely churn-bound). Even assuming a modest real-world save rate of 20% among customers who receive a retention offer as a direct result of a model flag — a figure consistent with published telecom win-back campaign performance — this scenario implies on the order of 2,766 additional retained customers per year, or approximately \$1.57 million in preserved annual revenue for this illustrative operator size, against a comparatively small cost of running the prediction pipeline and outreach campaigns.

Beyond direct revenue preservation, the feature importance analysis in Section 8.3 doubles as a prioritisation tool for the retention team: contract-upgrade incentives targeted at month-to-month subscribers, and bundled OnlineSecurity/TechSupport offers targeted at customers currently lacking them, are the two interventions the model's own importance ranking suggests will have the largest marginal effect on churn — allowing a fixed retention budget to be allocated to the segments where it is statistically most likely to pay off, rather than spread evenly across the customer base.

Table 8.1: Model Performance Comparison

Model	Accuracy	Precision	Recall	F1-Score	ROC AUC
Random Forest	73.95%	60.71%	36.34%	45.47%	76.51%
Gradient Boosting	73.81%	58.13%	44.18%	50.20%	76.39%
Logistic Regression	75.51%	62.10%	46.32%	53.06%	78.31%

Table 8.x: Confusion Matrix — Random Forest

	Predicted No	Predicted Yes	Total
Actual No	889	99	988
Actual Yes	268	153	421
Total	1157	252	1409

Table 8.x: Confusion Matrix — Gradient Boosting

	Predicted No	Predicted Yes	Total
Actual No	854	134	988
Actual Yes	235	186	421
Total	1089	320	1409

Table 8.x: Confusion Matrix — Logistic Regression

	Predicted No	Predicted Yes	Total
Actual No	869	119	988
Actual Yes	226	195	421
Total	1095	314	1409

8.6 Best Model Selection

Based on the ROC AUC metric, **Logistic Regression** was selected as the best-performing model with a ROC AUC score of **78.31%**. This model achieved accuracy of 75.51%, precision of 62.10%, recall of 46.32%, and F1-score of 53.06%. The model was serialized using Python's pickle module and deployed in the production Flask application.

Table 8.5: Top 15 Feature Importances

Rank	Feature	Importance
1	Contract_Two year	0.8577
2	Contract_One year	0.7337
3	tenure	0.5980
4	InternetService_Fiber optic	0.3171
5	PaymentMethod_Electronic check	0.2728
6	OnlineSecurity_Yes	0.2235
7	TechSupport_Yes	0.1672
8	TotalCharges	0.1380
9	SeniorCitizen	0.0940
10	PaperlessBilling	0.0708
11	MonthlyCharges	0.0554
12	StreamingMovies_Yes	0.0447
13	InternetService_No	0.0442
14	OnlineSecurity_No internet service	0.0442
15	OnlineBackup_No internet service	0.0442

8.7 Interpretation of Results

The results reveal several important business insights for telecommunications companies seeking to reduce customer churn. The strong importance of contract type features (Two year: 0.8577, One year: 0.7337) confirms that long-term contracts are the most effective mechanism for customer retention. Customers on month-to-month contracts represent the highest churn risk and should be prioritized for retention campaigns offering contract upgrades with attractive incentives.

Tenure (0.5980) as the third most important feature highlights that newer customers are significantly more likely to churn. This suggests that the onboarding experience and early engagement are critical periods for customer retention. Companies should implement targeted onboarding programs and regular check-ins during the first year of service.

The importance of InternetService_Fiber optic (0.3171) and PaymentMethod_Electronic check (0.2728) suggests that certain service and billing configurations are associated with higher churn. Fiber optic customers may experience higher churn due to premium pricing or service quality issues, while electronic check users may lack the automatic payment convenience that reduces friction and promotes retention.

The significance of OnlineSecurity (0.2235) and TechSupport (0.1672) as churn predictors indicates that customers without these value-added services are more likely to leave. Offering bundled security and support packages could serve as an effective retention strategy, simultaneously increasing average revenue per user (ARPU) and reducing churn risk.

CHAPTER 9

DEPLOYMENT

9.1 Local Deployment

The ChurnGuard AI system is designed for straightforward local deployment. The deployment process involves three steps: (1) Install Python dependencies using `pip install -r requirements.txt`, which installs Flask, pandas, NumPy, scikit-learn, Plotly, and Gunicorn; (2) Train the model using `python train_model.py`, which generates the dataset, trains classifier models, and saves artifacts; (3) Start the application using `python app.py`, which loads the model and starts the Flask development server on `http://127.0.0.1:5000`.

The application includes an auto-training feature: if no pre-trained model is found at startup, it automatically invokes the training pipeline before starting the server. This ensures that the application is always ready for predictions, even during initial deployment. Users access the system through a web browser and authenticate with the default credentials (`admin/admin123`).

9.2 Production Deployment Considerations

For production deployment, the application includes Gunicorn 22.0.0 in its requirements, a production-grade WSGI HTTP server for Python web applications. Production deployment would involve configuring environment variables for the secret key, setting up a reverse proxy (e.g., Nginx), enabling HTTPS with SSL certificates, implementing proper logging and monitoring, and configuring database-backed session storage for scalability.

The modular architecture of the application facilitates containerized deployment using Docker. A Dockerfile would specify the Python base image, install dependencies, copy application files, and configure Gunicorn as the entrypoint. Cloud deployment options include AWS Elastic Beanstalk, Google App Engine, Heroku, and Azure App Service, all of which support Python Flask applications natively.

9.3 Security Considerations

The current implementation prioritises demonstrating the machine learning pipeline and is not hardened for public production deployment; this section documents the known gaps candidly, as awareness of them is itself part of a responsible engineering assessment. The administrator credentials (`admin/admin123`) and the Flask `SECRET_KEY` are both hardcoded with plaintext fallback values in `config.py`. In any real deployment, both must be supplied

exclusively through environment variables or a secrets manager (e.g., AWS Secrets Manager, HashiCorp Vault), never committed to source control, and the default password must be replaced with a strong, unique credential enforced through a proper user-management system rather than a single shared account.

Session cookies are currently issued with Flask's default configuration. Production deployment should explicitly set `SESSION_COOKIE_SECURE=True` (cookies sent only over HTTPS), `SESSION_COOKIE_HTTPONLY=True` (inaccessible to client-side JavaScript, mitigating XSS-based session theft), and `SESSION_COOKIE_SAMESITE='Lax'` or `'Strict'` (mitigating cross-site request forgery). Since the application currently runs on plain HTTP during local development, all traffic — including the login form submitting credentials — is unencrypted; a production deployment must terminate TLS at a reverse proxy such as Nginx or a managed load balancer in front of Gunicorn.

The single-account authentication model also means the system provides no audit trail of who viewed which customer's prediction, which would be a compliance concern for real customer PII. A production-grade version would need per-user accounts with role-based access control, request logging, and rate limiting on both the `/login` route (to mitigate credential brute-forcing) and the `/api/chart-data/` endpoints (to mitigate scraping of the full dataset through repeated automated calls). These gaps are listed explicitly in Section 10.2 (Limitations) and informed the future-work item calling for OAuth 2.0 and role-based access control in Section 10.3.

CHAPTER 10

CONCLUSION AND FUTURE WORK

10.1 Summary of Findings

This project successfully developed and deployed ChurnGuard AI, a comprehensive customer churn prediction system that combines machine learning with an intuitive web interface. The system processes 7,043 customer records across 21 features, trains three classification models, and deploys the best-performing model for real-time predictions. Key findings include: (1) Logistic Regression achieved the best ROC AUC of 0.7831 among the three models tested; (2) Contract type, tenure, and internet service type are the most influential predictors of churn; (3) Month-to-month contracts, fiber optic service, and electronic check payment method are associated with significantly higher churn rates.

The web application successfully delivers five functional modules—dashboard, prediction, analytics, dataset explorer, and methodology—through a modern, professional dark-themed interface with interactive visualizations and real-time model inference capabilities.

10.2 Limitations

- The dataset is synthetically generated and may not capture all the complexities and nuances of real-world telecom customer data.
- The model performance (ROC AUC: 0.7831) could potentially be improved with more sophisticated feature engineering and hyperparameter tuning.
- The authentication system uses hardcoded credentials, which is not suitable for production multi-user environments.
- The system does not incorporate temporal patterns or time-series analysis of customer behavior.
- The file-based storage approach limits scalability for large-scale deployments.

10.3 Future Work

- Integrate real-world Telco Customer Churn datasets from Kaggle and other sources for model validation and comparison.
- Implement advanced algorithms including XGBoost, LightGBM, and deep learning models (Neural Networks) for improved predictive performance.
- Add SHAP (SHapley Additive exPlanations) analysis for individual prediction explanations and model interpretability.
- Implement real-time data streaming and batch processing capabilities for continuous model updates.
- Develop a comprehensive user management system with role-based access control and OAuth 2.0 integration.
- Add automated model retraining pipelines with performance monitoring and drift detection.
- Implement A/B testing frameworks for evaluating the effectiveness of retention strategies informed by model predictions.

- Create RESTful API endpoints for integration with external CRM and customer support systems.

10.4 Conclusion

The ChurnGuard AI Customer Churn Prediction System demonstrates the practical application of machine learning in solving real-world business problems. By combining robust predictive modeling with an accessible web interface, the system bridges the gap between data science and business decision-making, enabling telecommunications companies to proactively identify and retain at-risk customers. The project serves as a complete reference implementation of an end-to-end ML system, from data preprocessing through deployment, and provides a solid foundation for future enhancements and real-world applications.

REFERENCES

- [1] Ahmad, A. K., Jafar, A., and Aljoumaa, K. (2019). Customer churn prediction in telecom using machine learning in big data platform. *Journal of Big Data*, 6(1), 28.
- [2] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [3] Chawla, N. V., Bowyer, K. W., Hall, L. O., and Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16, 321-357.
- [4] Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232.
- [5] Hadden, J., Tiwari, A., Roy, R., and Ruta, D. (2007). Computer assisted customer churn management: State-of-the-art and future trends. *Computers & Operations Research*, 34(10), 2902-2917.
- [6] Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning*. Springer Series in Statistics.
- [7] IBM. (2019). Telco Customer Churn Dataset. IBM Developer. Available at: <https://developer.ibm.com/patterns/predict-customer-churn-using-watson-studio-and-jupyter-notebooks/>
- [8] Lundberg, S. M., and Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. *Advances in Neural Information Processing Systems*, 30, 4765-4774.
- [9] Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [10] Ribeiro, M. T., Singh, S., and Guestrin, C. (2016). "Why Should I Trust You?": Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135-1144.
- [11] Ronacher, A. (2010). Flask: A Python Microframework. Available at: <https://flask.palletsprojects.com/>
- [12] Vafeiadis, T., Diamantaras, K. I., Sarigiannidis, G., and Chatzisavvas, K. C. (2015). A comparison of machine learning techniques for customer churn prediction. *Simulation Modelling Practice and Theory*, 55, 1-9.
- [13] Van Rossum, G., and Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace.
- [14] Zheng, A., and Casari, A. (2018). *Feature Engineering for Machine Learning*. O'Reilly Media.

APPENDIX A

SOURCE CODE LISTINGS

A.1 Configuration Module (config.py)

```
import os

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY', 'churnprediction-secret-key-2026')
    DATA_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'data')
    MODEL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), 'models')

    # Default admin credentials
    ADMIN_USERNAME = 'admin'
    ADMIN_PASSWORD = 'admin123'
```

A.2 Model Training Pipeline (train_model.py)

```

"""
Customer Churn Prediction - Model Training Pipeline
Uses the Telco Customer Churn dataset to train a Random Forest classifier.
Generates the dataset if not present and saves the trained model.
"""

import os
import json
import pickle
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, confusion_matrix, classification_report
)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DATA_DIR = os.path.join(BASE_DIR, 'data')
MODEL_DIR = os.path.join(BASE_DIR, 'models')

os.makedirs(DATA_DIR, exist_ok=True)
os.makedirs(MODEL_DIR, exist_ok=True)

def generate_synthetic_telco_data(n_samples=7043):
    """Generate a synthetic Telco Customer Churn dataset."""
    np.random.seed(42)

    customer_ids = [f'{i:04d}-{"".join(np.random.choice(list("ABCDEFGHIJKLMNOPQRSTUVWXYZ"), 5))}' for i in range(n_samples)]

    gender = np.random.choice(['Male', 'Female'], n_samples)
    senior_citizen = np.random.choice([0, 1], n_samples, p=[0.84, 0.16])
    partner = np.random.choice(['Yes', 'No'], n_samples, p=[0.48, 0.52])
    dependents = np.random.choice(['Yes', 'No'], n_samples, p=[0.30, 0.70])

    tenure = np.random.choice(range(0, 73), n_samples)

    phone_service = np.random.choice(['Yes', 'No'], n_samples, p=[0.90, 0.10])
    multiple_lines = np.where(phone_service == 'No', 'No phone service',
                               np.random.choice(['Yes', 'No'], n_samples, p=[0.42, 0.58]))

    internet_service = np.random.choice(['DSL', 'Fiber optic', 'No'], n_samples, p=[0.34, 0.44, 0.22])

    online_security = np.where(internet_service == 'No', 'No internet service',
                               np.random.choice(['Yes', 'No'], n_samples, p=[0.35, 0.65]))
    online_backup = np.where(internet_service == 'No', 'No internet service',
                              np.random.choice(['Yes', 'No'], n_samples, p=[0.37, 0.63]))
    device_protection = np.where(internet_service == 'No', 'No internet service',
                                   np.random.choice(['Yes', 'No'], n_samples, p=[0.34, 0.66]))
    tech_support = np.where(internet_service == 'No', 'No internet service',
                             np.random.choice(['Yes', 'No'], n_samples, p=[0.33, 0.67]))
    streaming_tv = np.where(internet_service == 'No', 'No internet service',
                             np.random.choice(['Yes', 'No'], n_samples, p=[0.38, 0.62]))

```



```

streaming_movies = np.where(internet_service == 'No', 'No internet service',
                             np.random.choice(['Yes', 'No'], n_samples, p=[0.39, 0.61]))

contract = np.random.choice(['Month-to-month', 'One year', 'Two year'], n_samples, p=[0.55, 0.21,
paperless_billing = np.random.choice(['Yes', 'No'], n_samples, p=[0.59, 0.41])
payment_method = np.random.choice([
    'Electronic check', 'Mailed check', 'Bank transfer (automatic)', 'Credit card (automatic)'
], n_samples, p=[0.34, 0.23, 0.22, 0.21])

# Monthly charges correlated with services
base_charge = 20
monthly_charges = base_charge + np.random.normal(0, 5, n_samples)
monthly_charges += np.where(internet_service == 'Fiber optic', 40,
                             np.where(internet_service == 'DSL', 20, 0))
monthly_charges += np.where(online_security == 'Yes', 5, 0)
monthly_charges += np.where(online_backup == 'Yes', 5, 0)
monthly_charges += np.where(streaming_tv == 'Yes', 10, 0)
monthly_charges += np.where(streaming_movies == 'Yes', 10, 0)
monthly_charges = np.round(np.clip(monthly_charges, 18.25, 118.75), 2)

total_charges = np.round(monthly_charges * tenure + np.random.normal(0, 50, n_samples), 2)
total_charges = np.clip(total_charges, 18.8, 8684.8)

# Churn probability based on features
churn_prob = 0.15
churn_prob_arr = np.full(n_samples, churn_prob)
churn_prob_arr += np.where(contract == 'Month-to-month', 0.25, -0.10)
churn_prob_arr += np.where(internet_service == 'Fiber optic', 0.10, -0.05)
churn_prob_arr += np.where(tenure < 12, 0.15, np.where(tenure > 48, -0.15, 0.0))
churn_prob_arr += np.where(payment_method == 'Electronic check', 0.08, -0.02)
churn_prob_arr += np.where(online_security == 'No', 0.05, -0.03)
churn_prob_arr += np.where(tech_support == 'No', 0.05, -0.03)
churn_prob_arr += np.where(senior_citizen == 1, 0.05, 0.0)
churn_prob_arr = np.clip(churn_prob_arr, 0.02, 0.95)

churn = np.where(np.random.random(n_samples) < churn_prob_arr, 'Yes', 'No')

df = pd.DataFrame({
    'customerID': customer_ids,
    'gender': gender,
    'SeniorCitizen': senior_citizen,
    'Partner': partner,
    'Dependents': dependents,
    'tenure': tenure,
    'PhoneService': phone_service,
    'MultipleLines': multiple_lines,
    'InternetService': internet_service,
    'OnlineSecurity': online_security,
    'OnlineBackup': online_backup,
    'DeviceProtection': device_protection,
    'TechSupport': tech_support,
    'StreamingTV': streaming_tv,
    'StreamingMovies': streaming_movies,
    'Contract': contract,
    'PaperlessBilling': paperless_billing,
    'PaymentMethod': payment_method,
    'MonthlyCharges': monthly_charges,
    'TotalCharges': total_charges,
    'Churn': churn
})

```

```

    return df

def preprocess_data(df):
    """Preprocess the dataset for model training."""
    df = df.copy()
    df = df.drop('customerID', axis=1)

    df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
    df['TotalCharges'].fillna(df['TotalCharges'].median(), inplace=True)

    # Encode binary columns
    binary_cols = ['gender', 'Partner', 'Dependents', 'PhoneService', 'PaperlessBilling', 'Churn']
    le = LabelEncoder()
    for col in binary_cols:
        df[col] = le.fit_transform(df[col])

    # One-hot encode multi-category columns
    multi_cols = ['MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup',
                  'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
                  'Contract', 'PaymentMethod']
    df = pd.get_dummies(df, columns=multi_cols, drop_first=True)

    return df

def train_and_evaluate():
    """Train models and save the best one."""
    print("=" * 60)
    print(" CUSTOMER CHURN PREDICTION - MODEL TRAINING PIPELINE")
    print("=" * 60)

    # Load or generate data
    data_path = os.path.join(DATA_DIR, 'telco_churn.csv')
    if os.path.exists(data_path):
        print("\n[1/5] Loading existing dataset...")
        df = pd.read_csv(data_path)
    else:
        print("\n[1/5] Generating synthetic Telco Customer Churn dataset...")
        df = generate_synthetic_telco_data()
        df.to_csv(data_path, index=False)
        print(f" ✓ Dataset saved to {data_path}")

    print(f" ✓ Dataset shape: {df.shape}")
    print(f" ✓ Churn distribution:\n{df['Churn'].value_counts().to_string()}")

    # Save raw dataset stats
    stats = {
        'total_customers': len(df),
        'churn_count': int(df['Churn'].value_counts().get('Yes', 0)),
        'no_churn_count': int(df['Churn'].value_counts().get('No', 0)),
        'churn_rate': round(float(df['Churn'].value_counts(normalize=True).get('Yes', 0)) * 100, 2),
        'avg_tenure': round(float(df['tenure'].mean()), 2),
        'avg_monthly_charges': round(float(df['MonthlyCharges'].mean()), 2),
        'features': list(df.columns),
        'gender_distribution': df['gender'].value_counts().to_dict(),
        'contract_distribution': df['Contract'].value_counts().to_dict(),
        'internet_service_distribution': df['InternetService'].value_counts().to_dict(),
        'payment_method_distribution': df['PaymentMethod'].value_counts().to_dict(),
    }

```

```

# Preprocess

print("\n[2/5] Preprocessing data...")
df_processed = preprocess_data(df)

X = df_processed.drop('Churn', axis=1)
y = df_processed['Churn']

feature_names = list(X.columns)

# Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y
)
print(f" ✓ Training set: {X_train.shape[0]} samples")
print(f" ✓ Test set: {X_test.shape[0]} samples")
print(f" ✓ Features: {X_train.shape[1]}")

# Train models
print("\n[3/5] Training models...")

models = {
    'Random Forest': RandomForestClassifier(
        n_estimators=200, max_depth=15, min_samples_split=5,
        min_samples_leaf=2, random_state=42, n_jobs=-1
    ),
    'Gradient Boosting': GradientBoostingClassifier(
        n_estimators=150, max_depth=5, learning_rate=0.1,
        min_samples_split=5, random_state=42
    ),
    'Logistic Regression': LogisticRegression(
        max_iter=1000, random_state=42, C=1.0
    )
}

results = {}
best_model = None
best_score = 0
best_name = ''

for name, model in models.items():
    print(f"\n Training {name}...")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[: , 1]

    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred)
    rec = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)
    roc = roc_auc_score(y_test, y_prob)
    cm = confusion_matrix(y_test, y_pred).tolist()

    results[name] = {
        'accuracy': round(acc, 4),
        'precision': round(prec, 4),
        'recall': round(rec, 4),

```

```

        'f1_score': round(f1, 4),
        'roc_auc': round(roc, 4),

        'confusion_matrix': cm
    }

    print(f"    Accuracy: {acc:.4f}")
    print(f"    Precision: {prec:.4f}")
    print(f"    Recall: {rec:.4f}")
    print(f"    F1 Score: {f1:.4f}")
    print(f"    ROC AUC: {roc:.4f}")

    if roc > best_score:
        best_score = roc
        best_model = model
        best_name = name

print(f"\n ★ Best Model: {best_name} (ROC AUC: {best_score:.4f})")

# Feature importance
print("\n[4/5] Analyzing feature importance...")
if hasattr(best_model, 'feature_importances_'):
    importances = best_model.feature_importances_
else:
    importances = np.abs(best_model.coef_[0])

feature_importance = sorted(
    zip(feature_names, importances),
    key=lambda x: x[1], reverse=True
)

top_features = [{'feature': f, 'importance': round(float(i), 4)} for f, i in feature_importance[:10]]

print(" Top 10 Features:")
for i, (feat, imp) in enumerate(feature_importance[:10], 1):
    print(f"    {i:2d}. {feat:<35s} {imp:.4f}")

# Save everything
print("\n[5/5] Saving model and artifacts...")

pickle.dump(best_model, open(os.path.join(MODEL_DIR, 'churn_model.pkl'), 'wb'))
pickle.dump(scaler, open(os.path.join(MODEL_DIR, 'scaler.pkl'), 'wb'))
pickle.dump(feature_names, open(os.path.join(MODEL_DIR, 'feature_names.pkl'), 'wb'))

stats['model_results'] = results
stats['best_model'] = best_name
stats['top_features'] = top_features

with open(os.path.join(DATA_DIR, 'stats.json'), 'w') as f:
    json.dump(stats, f, indent=2)

print(f" ✓ Model saved: churn_model.pkl")
print(f" ✓ Scaler saved: scaler.pkl")
print(f" ✓ Feature names saved: feature_names.pkl")
print(f" ✓ Stats saved: stats.json")

print("\n" + "=" * 60)
print(" TRAINING COMPLETE!")
print("\n" + "=" * 60)

```

```
    return best_model, scaler, feature_names, stats

if __name__ == '__main__':
    train_and_evaluate()
```



```

from sklearn.preprocessing import LabelEncoder
binary_cols = ['gender', 'Partner', 'Dependents', 'PhoneService', 'PaperlessBilling']
le = LabelEncoder()
for col in binary_cols:
    input_df[col] = le.fit_transform(input_df[col])

multi_cols = ['MultipleLines', 'InternetService', 'OnlineSecurity', 'OnlineBackup',
              'DeviceProtection', 'TechSupport', 'StreamingTV', 'StreamingMovies',
              'Contract', 'PaymentMethod']
input_df = pd.get_dummies(input_df, columns=multi_cols, drop_first=True)

# Ensure all features are present
for feat in feature_names:
    if feat not in input_df.columns:
        input_df[feat] = 0
input_df = input_df[feature_names]

X_input = scaler.transform(input_df.values)
pred = model.predict(X_input)[0]
prob = model.predict_proba(X_input)[0]

prediction = 'Churn' if pred == 1 else 'No Churn'
probability = round(float(prob[1]) * 100, 2)

return render_template('predict.html', prediction=prediction, probability=probability, form_data=f)

@app.route('/analytics')
@login_required
def analytics():
    return render_template('analytics.html', stats=stats)

@app.route('/dataset')
@login_required
def dataset():
    page = int(request.args.get('page', 1))
    per_page = 20

    if df is not None:
        total = len(df)
        total_pages = (total + per_page - 1) // per_page
        start = (page - 1) * per_page
        end = start + per_page

        data = df.iloc[start:end].to_dict('records')
        columns = list(df.columns)
    else:
        data = []
        columns = []
        total = 0
        total_pages = 0

    return render_template('dataset.html', data=data, columns=columns,
                          page=page, total_pages=total_pages, total=total)

@app.route('/methodology')
@login_required

```



```

def methodology():

    return render_template('methodology.html', stats=stats)

@app.route('/api/chart-data/&lt;chart_type&gt;')
@login_required
def chart_data(chart_type):
    """API endpoint for dynamic chart data."""
    if df is None:
        return jsonify({'error': 'No data available'}), 404

    if chart_type == 'churn_distribution':
        counts = df['Churn'].value_counts()
        return jsonify({
            'labels': counts.index.tolist(),
            'values': counts.values.tolist()
        })

    elif chart_type == 'tenure_distribution':
        churned = df[df['Churn'] == 'Yes']['tenure']
        not_churned = df[df['Churn'] == 'No']['tenure']
        return jsonify({
            'churned': churned.tolist(),
            'not_churned': not_churned.tolist()
        })

    elif chart_type == 'contract_churn':
        cross = pd.crosstab(df['Contract'], df['Churn'])
        return jsonify({
            'labels': cross.index.tolist(),
            'churn_yes': cross.get('Yes', pd.Series([0]*len(cross))).tolist(),
            'churn_no': cross.get('No', pd.Series([0]*len(cross))).tolist()
        })

    elif chart_type == 'monthly_charges':
        churned = df[df['Churn'] == 'Yes']['MonthlyCharges']
        not_churned = df[df['Churn'] == 'No']['MonthlyCharges']
        return jsonify({
            'churned': churned.describe().to_dict(),
            'not_churned': not_churned.describe().to_dict()
        })

    elif chart_type == 'internet_service':
        cross = pd.crosstab(df['InternetService'], df['Churn'])
        return jsonify({
            'labels': cross.index.tolist(),
            'churn_yes': cross.get('Yes', pd.Series([0]*len(cross))).tolist(),
            'churn_no': cross.get('No', pd.Series([0]*len(cross))).tolist()
        })

    elif chart_type == 'payment_method':
        cross = pd.crosstab(df['PaymentMethod'], df['Churn'])
        return jsonify({
            'labels': cross.index.tolist(),
            'churn_yes': cross.get('Yes', pd.Series([0]*len(cross))).tolist(),
            'churn_no': cross.get('No', pd.Series([0]*len(cross))).tolist()
        })

    elif chart_type == 'feature_importance':

```

```

        if stats.get('top_features'):
            features = stats['top_features']

            return jsonify({
                'labels': [f['feature'] for f in features],
                'values': [f['importance'] for f in features]
            })

    elif chart_type == 'model_comparison':
        if stats.get('model_results'):
            results = stats['model_results']
            return jsonify({
                'models': list(results.keys()),
                'accuracy': [results[m]['accuracy'] for m in results],
                'precision': [results[m]['precision'] for m in results],
                'recall': [results[m]['recall'] for m in results],
                'f1_score': [results[m]['f1_score'] for m in results],
                'roc_auc': [results[m]['roc_auc'] for m in results]
            })

    return jsonify({'error': 'Unknown chart type'}), 400

if __name__ == '__main__':
    if model is None:
        print("\n■ Model not found. Training model first...\n")
        from train_model import train_and_evaluate
        train_and_evaluate()
        model, scaler, feature_names, stats, df = load_artifacts()

    print("\n■ Starting Customer Churn Prediction System...")
    print("  Dashboard: http://127.0.0.1:5000")
    print("  Login: admin / admin123\n")
    app.run(debug=True, port=5000)

```

A.4 Report Styling & Configuration Module (report_config.py)

```

"""
Report configuration constants and shared utilities.
"""

import os
from reportlab.lib.pagesizes import A4
from reportlab.lib.colors import HexColor
from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
from reportlab.lib.enums import TA_CENTER, TA_JUSTIFY, TA_LEFT, TA_RIGHT

WIDTH, HEIGHT = A4
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
MARGIN = 60
CONTENT_WIDTH = WIDTH - 2 * MARGIN

# Color palette
COLORS = {
    'primary': HexColor('#1a1a2e'),
    'accent': HexColor('#6366f1'),
    'accent_light': HexColor('#818cf8'),
    'text': HexColor('#1e293b'),
    'text_light': HexColor('#64748b'),
    'heading': HexColor('#0f172a'),
    'white': HexColor('#ffffff'),
    'bg_light': HexColor('#f8fafc'),
    'border': HexColor('#e2e8f0'),
    'success': HexColor('#22c55e'),
    'danger': HexColor('#ef4444'),
    'warning': HexColor('#f59e0b'),
    'code_bg': HexColor('#f1f5f9'),
    'table_header': HexColor('#4338ca'),
    'table_alt': HexColor('#f1f5f9'),
}

# Project metadata
PROJECT = {
    'title': 'Customer Churn Prediction System',
    'subtitle': 'Using Machine Learning and Predictive Analytics',
    'author': 'Aryan Sharma',
    'year': '2026',
    'institution': '',
    'app_name': 'ChurnGuard AI',
}

# Screenshot paths (user-provided)
SCREENSHOT_DIR = os.path.join(BASE_DIR, 'screenshots')

def get_styles():
    """Return custom paragraph styles for the report."""
    styles = getSampleStyleSheet()

    styles.add(ParagraphStyle(
        'ReportTitle', parent=styles['Title'],
        fontSize=28, leading=34, textColor=COLORS['white'],
        alignment=TA_CENTER, spaceAfter=12, fontName='Helvetica-Bold',

```

```

))
styles.add(ParagraphStyle(
    'ReportSubtitle', parent=styles['Normal'],
    fontSize=14, leading=20, textColor=HexColor('#c7d2fe'),
    alignment=TA_CENTER, spaceAfter=6, fontName='Helvetica',
))
styles.add(ParagraphStyle(
    'ChapterTitle', parent=styles['Heading1'],
    fontSize=22, leading=28, textColor=COLORS['accent'],
    spaceBefore=20, spaceAfter=14, fontName='Helvetica-Bold',
))
styles.add(ParagraphStyle(
    'SectionTitle', parent=styles['Heading2'],
    fontSize=16, leading=22, textColor=COLORS['heading'],
    spaceBefore=16, spaceAfter=8, fontName='Helvetica-Bold',
))
styles.add(ParagraphStyle(
    'SubSection', parent=styles['Heading3'],
    fontSize=13, leading=18, textColor=COLORS['text'],
    spaceBefore=10, spaceAfter=6, fontName='Helvetica-Bold',
))
styles.add(ParagraphStyle(
    'BodyText2', parent=styles['Normal'],
    fontSize=11, leading=17, textColor=COLORS['text'],
    alignment=TA_JUSTIFY, spaceAfter=8, fontName='Helvetica',
    firstLineIndent=20,
))
styles.add(ParagraphStyle(
    'BodyNoIndent', parent=styles['Normal'],
    fontSize=11, leading=17, textColor=COLORS['text'],
    alignment=TA_JUSTIFY, spaceAfter=8, fontName='Helvetica',
))
styles.add(ParagraphStyle(
    'BulletStyle', parent=styles['Normal'],
    fontSize=11, leading=17, textColor=COLORS['text'],
    spaceAfter=4, fontName='Helvetica', leftIndent=30, bulletIndent=15,
))
styles.add(ParagraphStyle(
    'CodeStyle', parent=styles['Normal'],
    fontSize=9, leading=13, textColor=COLORS['text'],
    fontName='Courier', backgroundColor=COLORS['code_bg'],
    leftIndent=15, rightIndent=15, spaceBefore=6, spaceAfter=6,
    borderWidth=1, borderColor=COLORS['border'], borderPadding=8,
))
styles.add(ParagraphStyle(
    'Caption', parent=styles['Normal'],
    fontSize=9, leading=13, textColor=COLORS['text_light'],
    alignment=TA_CENTER, spaceAfter=12, fontName='Helvetica-Oblique',
))
styles.add(ParagraphStyle(
    'TOCEntry', parent=styles['Normal'],
    fontSize=12, leading=22, textColor=COLORS['text'],
    fontName='Helvetica', leftIndent=20,
))
styles.add(ParagraphStyle(
    'TOCChapter', parent=styles['Normal'],
    fontSize=13, leading=24, textColor=COLORS['heading'],
    fontName='Helvetica-Bold', leftIndent=0, spaceBefore=4,
))

```

```
styles.add(ParagraphStyle(  
    'PageNumber', parent=styles['Normal'],  
    fontSize=9, textColor=COLORS['text_light'],  
    alignment=TA_CENTER, fontName='Helvetica',  
))  
styles.add(ParagraphStyle(  
    'Footer', parent=styles['Normal'],  
    fontSize=8, textColor=COLORS['text_light'],  
    alignment=TA_CENTER, fontName='Helvetica',  
))  
  
return styles
```

APPENDIX B

GLOSSARY OF TERMS

AUC — Area Under the Curve — a measure of a classifier's ability to distinguish between classes.

Churn — The phenomenon where customers discontinue their service with a provider.

Classification — A supervised learning task where the output is a discrete category (e.g., Churn/No Churn).

Confusion Matrix — A table showing true positives, true negatives, false positives, and false negatives.

F1-Score — The harmonic mean of precision and recall, providing a balanced measure of classification quality.

Feature Engineering — The process of selecting, transforming, and creating features from raw data to improve model performance.

Flask — A lightweight Python web framework for building web applications and APIs.

Gradient Boosting — An ensemble technique that builds models sequentially, each correcting errors of the previous one.

KPI — Key Performance Indicator — a measurable value demonstrating effectiveness.

Label Encoding — Converting categorical text labels to numerical values (e.g., Male → 1, Female → 0).

Logistic Regression — A statistical model that uses the sigmoid function to predict binary outcomes.

MVC — Model-View-Controller — a software architecture pattern separating data, presentation, and logic.

One-Hot Encoding — Converting a categorical variable into multiple binary columns (one per category).

Precision — The proportion of positive predictions that are actually correct: $TP / (TP + FP)$.

Random Forest — An ensemble of decision trees trained on bootstrap samples with random feature subsets.

Recall — The proportion of actual positives correctly identified: $TP / (TP + FN)$. Also called Sensitivity.

ROC — Receiver Operating Characteristic — a curve plotting True Positive Rate vs False Positive Rate.

Scikit-learn — A popular Python library for machine learning, providing classification, regression, and clustering tools.

Standard Scaling — Transforming features to have zero mean and unit variance: $z = (x - \mu) / \sigma$.

WSGI — Web Server Gateway Interface — a specification for Python web application/server communication.